PROJECT PROPOSAL

BankChurn AI

A web application for predicting customer churn in retail banking

Item	Detail
Course	Software Engineering, MUST University
Student	Hedi _____ ID _____
Type	Individual project (70% of the final grade)
Methodology	Scrum, one-week sprints
Repository	github.com/Abdelhedi1/Bank-Churn-App2
Date	October 2026

1. Introduction and problem

Retail banks lose customers quietly. In the dataset used for this project, 20.4% of 8,000 customers had already left the bank. By the time a customer closes an account, it is usually too late to react. Relationship managers need to know early which customers are at risk, why the risk is high, and which action is reasonable. They must also be able to record what was done, and the bank must be able to show who looked at which customer data.

The project follows the Software Requirements Specification of Lab 2 (requirements REQ-01 to REQ-20) and the component design of Lab 5 (components C1 to C9).

2. Objectives

- Train two churn models on customer data and select the better one automatically.
- Score every active customer each week and store the results.
- Show risk on a dashboard with the main reasons behind each score.
- Raise alerts for customers above a threshold that the administrator controls.
- Suggest retention actions and record them. Nothing is ever sent to a customer automatically.
- Restrict access by role, mask customer identifiers and keep an audit log.

3. Scope

In scope: data upload, feature generation, model training and comparison, weekly scoring, dashboard, single-customer prediction, customer insights, analytics, retention actions, monthly KPI report and administration. **Out of scope:** sending messages to customers, e-mail alerts, production deployment and encryption set-up, and load testing.

4. Stakeholders and users

Role	Main needs
Relationship manager	See risk and reasons for own customers, receive alerts, log actions
Retention manager	See all customers, export high-risk lists, read the monthly report
Executive	Read-only view of key figures and reports
Administrator	Load data, train models, run scoring, manage users, read the audit log

5. Methodology

Scrum was chosen for three reasons. The team is one developer, so ceremonies stay light. The deadline is fixed, so a working increment at the end of each week lowers the risk of arriving with nothing to show. The requirements are already stable (SRS), so the main uncertainty is technical, and short sprints expose technical problems early. Kanban was considered and rejected because there is no continuous stream of change requests. Estimates use T-shirt sizes. Where AI helps draft estimates or sprint plans, the developer validates them.

Sprint	Focus	Planned deliverable
1	Data and model	8,000-customer dataset, features, two models compared with metrics
2	Backend	Data store, login and roles, scoring, alerts, JSON API, automated tests
3	Interface	Dashboard and six more pages following the reference design
4	Documents	UML diagrams, API description, user guide, ethics reflection, AI log, slides, demo

Definition of done: the feature works in the browser, automated tests pass, lint is clean, and the documents affected are updated.

6. Requirements: user stories and backlog

The stories were first drafted with an AI assistant from the SRS notes (prompt recorded in the AI usage log) and then edited by the developer.

ID	User story	Priority	Size
US1	As a relationship manager I see the churn probability, tier and main reasons for a customer	Must	M
US2	As a relationship manager I only see my own customers	Must	M
US3	As a retention manager I receive alerts for high-risk customers	Must	M
US4	As an administrator I upload data and train models; the best model becomes active	Must	L
US5	As an administrator I rely on automatic weekly scoring	Must	M
US8	As an auditor I can see who viewed customer data; ids are masked	Must	S
US6	As an executive I read a monthly KPI report	Should	S
US7	As a manager I log retention actions	Should	S
NFR3	Encryption in transit and at rest	Could	M

7. Architecture and technology

The system is one Flask application with three layers: a browser interface (seven pages, JavaScript, Plotly charts), a server (pages, JSON API, login and audit, ingestion, scoring with a weekly scheduler, model management) and storage (a SQLite database and trained model files). Design patterns: Strategy for interchangeable models, Repository for data access and Observer for alerts.

Layer	Technology
Interface	HTML templates, vanilla JavaScript, Plotly
Server	Python, Flask, Werkzeug security
Machine learning	pandas, scikit-learn, imbalanced-learn (SMOTE), LightGBM
Storage	SQLite, joblib model files
Quality	pytest and pytest-cov, ruff, GitHub Actions
Documents	UML in Mermaid, API in OpenAPI

8. Quality plan

- Every feature has automated tests; target coverage at least 60%, enforced in the CI pipeline.
- Lint with ruff on every push; complexity measured with radon.
- Small commits to the GitHub repository, with the history showing the work.
- Refactoring applied through reviewed changes, with before and after metrics.
- Documents: README, API description, user guide, architecture, ethics reflection.

9. Risks and mitigation

Risk	Likelihood	Mitigation
AI-generated code or text is wrong	Medium	Run everything, write tests, review each change, keep an AI usage log
The model treats groups of customers unfairly	Medium	Suggestions only, a person decides; measure fairness before real use
Scope too large for the deadline	Medium	Must stories first; Could items dropped
Customer data privacy	Low	Synthetic data, masked ids, role scoping, audit log
Performance not verified	Medium	Record scoring time per run; plan a load test as future work

10. Use of AI

An AI assistant (Claude) is used for coding, testing and drafting documents, following the cycle plan, prompt, review, test, document. Every use is listed in the AI usage log, with the prompt and a flag showing whether the output was used as is or edited. A reflection on risks, licences and intellectual property is in the ethics document. The student remains responsible for understanding and defending every part of the work.

11. Deliverables

- Source code with tests and a passing CI pipeline
- UML diagrams (images and Mermaid source), OpenAPI description
- User guide, developer documents, requirements traceability
- Refactor and optimization report with metrics
- AI usage log, ethics reflection, signed honesty statement
- Final presentation and recorded demonstration